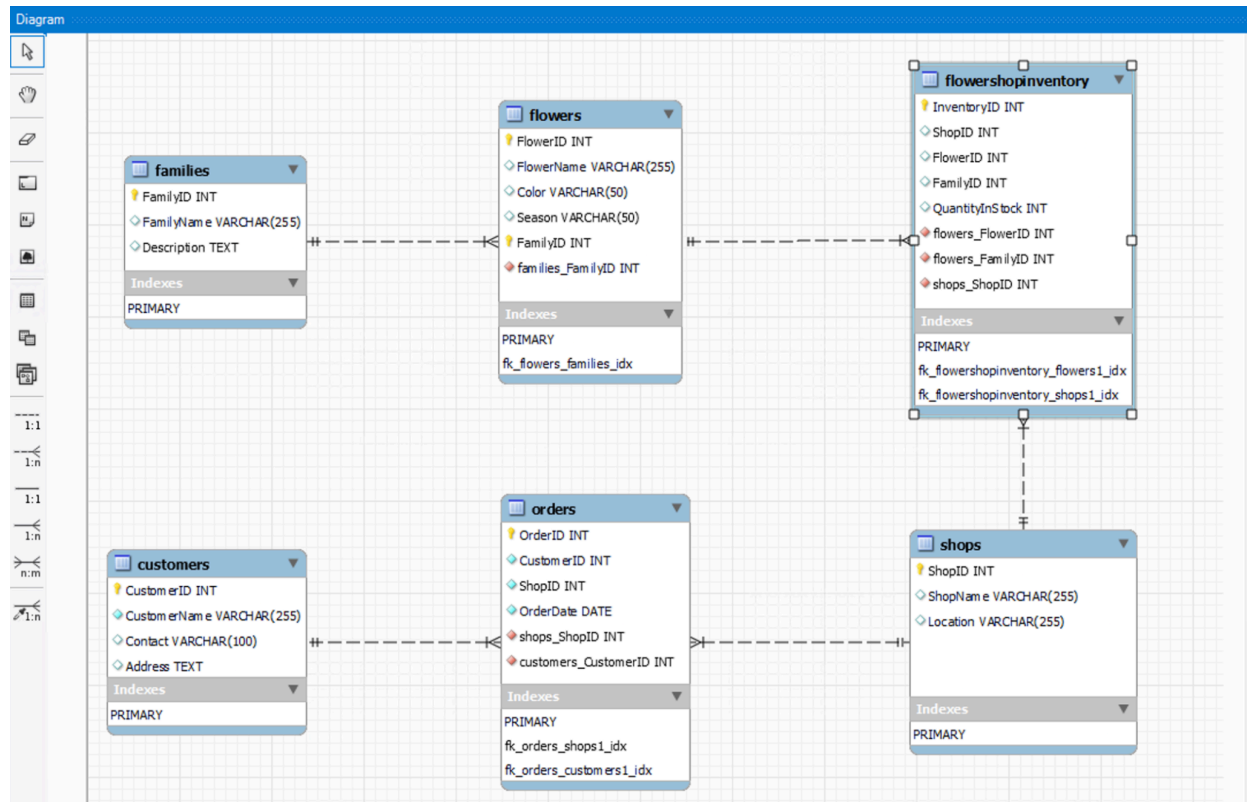


Part 1

Task 1: See Task_1_Model.mwb file

Task 2: See Task_2_Dump.sql file

Task 3:



- By using 1:N structures data is centralized. For example if a specific storefront's attributes are recorded exactly once in the shops table it allows millions of national transaction rows to point back to these singular records via foreign keys without duplicating data. This design prevents massive redundancy and synchronization errors ensuring that an update to a shop's location is made in only one place and instantly shown everywhere. Additionally since these relationships are non identifying child tables like orders maintain independent primary keys ensuring that historical sales records remain completely intact even if a storefront closes.
- An example scenario can be where an automated customer checkout system attempts to process an online purchase but a network error corrupts the incoming payload and assigns a non existent CustomerID to the transaction. A script tries to insert this new sales record into the orders table however since the main customers table does not contain this ID the database engine detects the missing parent record and immediately blocks the operation with a violation error.

Part 2

1.

Develop: See P2_1.sql for query

Execute:

The screenshot shows a database query tool interface. At the top, the query is written in SQL:

```
1 • Select FlowerID, FlowerName, Color, Season, FamilyID
2 From flowers
3 Where color <> 'Red' AND Season Not In ('Spring', 'Winter')
4 Limit 10;
```

Below the query, the results are displayed in a table with the following columns: FlowerID, FlowerName, Color, Season, and FamilyID. The results show 10 rows of data:

FlowerID	FlowerName	Color	Season	FamilyID
2	Sunflower	Yellow	Summer	1
6	Pea Flower	Purple	Summer	5
8	Wheat Flower	Brown	Autumn	7
9	Carrot Flower	White	Summer	8
11	Cactus Flower	Pink	Summer	10
12	Potato Flower	White	Summer	11
15	Palm Flower	Yellow	Summer	14
18	Mint Flower	White	Summer	17
20	Figwort	Purple	Summer	19
24	Asparagus Flower	Green	Summer	23

At the bottom of the screenshot, the 'Output' section shows the execution details:

#	Time	Action	Message	Duration / Fetch
3	02:19:38	Select FlowerID, FlowerName, Color, Season, FamilyID From flowers Where color <> 'Red' AND Season Not...	10 row(s) returned	0.016 sec / 0.000 sec

Diagnose: The query successfully returned the top 10 rows matching using the filters of the flower color not being red and not in the seasons of Spring and Winter. The data shows a concentration of Summer options such as Sunflower, Pea Flower, Carrot Flower, and Mint Flower as well as an Autumn option like Wheat Flower. There are a variety of vibrant non traditional shades like yellow, purple, brown, white, pink, and green. This shows that filtering out red, spring, and winter items leaves a diverse collection of warm weather flora.

Strategize: Since the current stock leans heavily toward Summer items in alternative shades like yellow, white, and purple, the business should target the off season market though some kind of event. The marketing teams can bundle these specific varieties into a "Sun Drenched Summer Event" or rustic "Autumn Harvest" campaign. This is to attempt to shift consumer attention away from standard red roses and to clear out the warm weather inventory before seasonal demand drops off.

2.

Develop: See P2_2.sql for query

Execute:

The screenshot shows a database query tool interface. At the top, there are tabs for 'FlowerDB', 'P2_1', and 'P2_2'. The 'P2_2' tab is active, displaying a SQL query:

```
1 Select f.FlowerID, f.FlowerName, f.Color, f.Season
2 From flowers f
3 Left Join flowershopinventory i ON f.FlowerID = i.FlowerID
4 Where i.FlowerID is Null
5 Limit 10;
```

Below the query editor, the 'Result Grid' is visible, showing a table with 4 columns: FlowerID, FlowerName, Color, and Season. The results are as follows:

FlowerID	FlowerName	Color	Season
33	Lupine	Purple	Spring
34	Snapdragon	Yellow	Summer
35	Marigold	Orange	Autumn
36	Petunia	Pink	Summer
37	Begonia	Red	Summer
38	Daffodil	Yellow	Spring
39	Chrysanthemum	White	Autumn
40	Zinnia	Multi-colored	Summer

At the bottom, the 'Output' section shows a message: '1 02:43:01 Select f.FlowerID, f.FlowerName, f.Color, f.Season From flowers f Left Join flowershopinventory i ON f.FlowerID = i.FlowerID 8 row(s) returned'. The duration is 0.032 sec / 0.000 sec.

Diagnose: The query output identifies exactly 8 flower varieties that are completely absent from the flowershopinventory table. While these products exist as catalog records in the main database not a single physical shop currently holds them in stock. The unstocked items span multiple seasons like Spring, Summer, and Autumn and color palettes.

Strategize: Management must immediately investigate why these 8 active catalog items carry zero marketplace availability. If these varieties are outdated or discontinued they should be archived to prevent systemic confusion. Otherwise if they represent active offerings procurement should prioritize sourcing these missing items to capture unrealized retail demand.

3.

Develop: See P2_3.sql for query

Execute:

The screenshot shows a database query execution interface. At the top, there are tabs for 'FlowerDB', 'P2_1', 'P2_2', and 'P2_3'. The 'P2_3' tab is active. Below the tabs, there is a toolbar with various icons and a 'Limit to 1000 rows' dropdown. The main area displays a SQL query:

```
1 • Select c.CustomerID, c.CustomerName, c.Contact, Count(Distinct o.ShopID) AS UniqueShopsVisited
2   From customers c
3   Inner Join orders o On c.CustomerID = o.CustomerID
4   Group By
5   c.CustomerID, c.CustomerName, c.Contact
6   Having Count(Distinct o.ShopID) >= 2
7   Limit 10;
```

Below the query, there is a 'Result Grid' section. It includes a 'Filter Rows' input, an 'Export' button, and a 'Wrap Cell Content' checkbox. The grid displays the following data:

CustomerID	CustomerName	Contact	UniqueShopsVisited
1	Alice Smith	alice@example.com	2
14	Noah Adams	noah@example.com	2
22	Victor Lee	victor@example.com	4
30	Diana Howard	diana@example.com	3
32	Fiona Edwards	fiona@example.com	5

On the right side of the interface, there is a vertical toolbar with icons for 'Result Grid', 'Form Editor', 'Field Types', and 'Query Stats'. At the bottom, there is a 'Result 1' section with an 'Output' tab and an 'Action Output' dropdown. The 'Output' tab shows the query execution details:

#	Time	Action	Message	Duration / Fetch
1	03:01:35	Select c.CustomerID, c.CustomerName, c.Contact, Count(Distinct o.ShopID) AS UniqueShopsVisited From cu...	5 row(s) returned	0.015 sec / 0.000 sec

Diagnose: The query output there are exactly 5 multi location customers who actively cross shop between storefronts. In particular clients like Victor Lee and Fiona Edwards show an exceptionally broad geographic footprint by purchasing from 4 and 5 separate shop locations respectively. This data confirms that the brand has strong loyalty and mobility in these individuals as they are traveling across multiple shops instead of a single neighborhood store.

Strategize: Marketing should immediately transition these 5 buyers into a premium tier of the brand loyalty program to capitalize on their multi site habits. Because they already travel between physical retail locations sending them multistore mobile perks or rewards will incentivize them to keep visiting new branches. The customer experience teams should also interview outliers like Fiona Edwards to identify what drives their cross city shopping behavior helping to refine marketing strategies for more casual shoppers.

4.

Develop: See P2_4.sql for query

Execute:

The screenshot displays a database management tool interface. At the top, there are tabs for 'FlowerDB', 'P2_1', 'P2_2', 'P2_3', and 'P2_4'. The 'P2_4' tab is active, showing a SQL query in a text editor. The query is as follows:

```
1 • Select f.FamilyID, fam.FamilyName, ROUND(AVG(f.TotalStockPerShop), 2) As CrossShopAverageStock
2 From (
3   Select FamilyID, ShopID, SUM(QuantityInStock) AS TotalStockPerShop
4   From flowershopinventory
5   Group By FamilyID, ShopID
6 ) f
7 Inner Join families fam ON f.FamilyID = fam.FamilyID
8 Group By
9   f.FamilyID,
10  fam.FamilyName
11 Having AVG(f.TotalStockPerShop) >= 140
12 Limit 10;
```

Below the query editor, the 'Result Grid' is visible, showing the results of the query. The grid has three columns: 'FamilyID', 'FamilyName', and 'CrossShopAverageStock'. The results are as follows:

FamilyID	FamilyName	CrossShopAverageStock
19	Scrophulariaceae	140.00
25	Apocynaceae	140.00
26	Araceae	145.00
27	Commelinaceae	150.00
28	Ericaceae	160.00
30	Geraniaceae	160.00

At the bottom of the interface, there is an 'Output' section with a table showing the execution details:

#	Time	Action	Message	Duration / Fetch
1	03:23:22	Select f.FamilyID, fam.FamilyName, ROUND(AVG(f.TotalStockPerShop), 2) As CrossShopAverageStock From ...	6 row(s) returned	0.015 sec / 0.000 sec

Diagnose: The query output identifies exactly 6 flower families that successfully meet the high volume criteria across the business. These results confirm that these specific botanical groups maintain a stable baseline of at least 140 consolidated units per store.

Strategize: Supply chain leadership should designate these 6 high volume families as core, predictable assets for automated inventory replenishment algorithms. Since these varieties maintain massive baseline stock levels across all the stores, corporate procurement can confidently negotiate long term bulk contracts with commercial growers to reduce per unit costs.

5.

Develop: See P2_5.sql for query

Execute:

The screenshot displays a SQL query execution environment. The query is as follows:

```
1 • Select s.ShopID, s.ShopName, s.Location, MIN(i.QuantityInStock) AS LowestStockQuantity
2 From shops s
3 Inner Join flowershopinventory i On s.ShopID = i.ShopID
4 Group By s.ShopID, s.ShopName, s.Location
5 Having MIN(i.QuantityInStock) > 100
6 Limit 10;
```

The results are shown in a table with the following data:

ShopID	ShopName	Location	LowestStockQuantity
10	Carnation Corner	Saskatoon	105
11	Violet Villa	Victoria	115
12	Borage Bungalow	Regina	125
13	Iris Inn	St. John's	135
14	Primrose Place	Charlottetown	145
15	Asparagus Alcove	Fredericton	155
16	Olive Outset	Yellowknife	165

The interface also shows a status bar at the bottom indicating the query was executed successfully, returning 7 rows in 0.031 seconds.

Diagnose: The query output lists exactly 7 storefronts Carnation Corner, Violet Villa, and Borage Bungalow that currently hold over 100 units for every single item they carry. Because the filtering mechanism checks the absolute lowest stock value for each location these results confirm that these specific regional branches are entirely free from immediate low inventory risks.

Strategize: Operations leadership should temporarily pause active inventory replenishment shipments to these 7 over supplied locations to prevent warehousing bottlenecks and bloated holding costs. Since their lowest stocked items still exceed 100 units, capital and logistics resources should be diverted toward under supplied stores. Additionally regional managers for these high volume branches can confidently run aggressive local promotions or transfer excess stock to nearby storefronts to optimize the overall network distribution.

6.

Develop: See P2_6.sql for query

Execute:

The screenshot shows a database query editor interface. At the top, there are tabs for different queries: FlowerDB, P2_1, P2_2, P2_3, P2_4, P2_5, and P2_6 (which is the active tab). The SQL query is as follows:

```
1 Select f.FamilyID, f.FamilyName, IfNull(Sum(i.QuantityInStock), 0) As TotalEnterpriseStock
2 From families f
3 Left Join flowerShopInventory i On f.FamilyID = i.FamilyID
4 Group By f.FamilyID, f.FamilyName
5 Limit 10;
```

Below the query editor, the 'Result Grid' is displayed, showing the results of the query. The grid has three columns: FamilyID, FamilyName, and TotalEnterpriseStock. The results are as follows:

FamilyID	FamilyName	TotalEnterpriseStock
1	Asteraceae	35
2	Liliaceae	25
3	Rosaceae	30
4	Orchidaceae	40
5	Fabaceae	35
6	Ranunculaceae	50
7	Poaceae	45
8	Aplacae	60
9	Brassicaceae	55
10	Cactaceae	65

At the bottom of the interface, there is an 'Output' section showing the execution details. It includes a table with columns: #, Time, Action, Message, and Duration / Fetch. The output shows that the query was executed successfully at 04:02:18, returning 10 rows in 0.031 seconds.

Diagnose: The query output provides a high level overview of total organization wide holdings of the first 10 main flower families. The data shows low inventory volumes across the board ranging from a minimum of 25 units for Liliaceae to a peak of only 65 units for Cactaceae. This uniformly thin distribution confirms that none of these core categories are being overstocked at the enterprise level.

Strategize: Procurement teams must immediately evaluate if these stock totals are sufficient to support sustained consumer demand across the entire national shop footprint. Since flagship categories like Rosaceae and Liliaceae are holding low company wide numbers the business may be at risk during sudden market spikes. Supply chain analysts should immediately set up automated minimum threshold alerts for these main families.

7.

Business Critical Question: Which specific retail shop locations are generating the highest volume of order transactions despite maintaining a below average inventory stock?

Develop: See P2_7.sql for query

Execute:

```
1 Select s.ShopID, s.ShopName, s.Location, Count(Distinct o.OrderID) As TotalOrdersPlaced, IFNULL(Sum(i.QuantityInStock),0) As TotalCurrentStock
2 From shops s
3 Inner Join orders o on s.ShopID = o.ShopID
4 Left Join flowerShopInventory i On s.ShopID = i.ShopID
5 Group by s.ShopID, s.ShopName, s.Location
6 Having IFNULL(Sum(i.QuantityInStock), 0) < (
7     Select Avg(ShopTotalStock)
8     From (
9         Select SUM(QuantityInStock) As ShopTotalStock
10        From flowerShopInventory
11        Group By ShopID
12    ) As sub
13 )
14 Order By TotalOrdersPlaced Desc
15 Limit 10;
```

ShopID	ShopName	Location	TotalOrdersPlaced	TotalCurrentStock
1	Floral Fantasy	Toronto	2	70
2	Blossom Boutique	Vancouver	2	110
3	Petal Palace	Montreal	2	150
17	Dogbane Den	Whitehorse	1	0
18	Arum Attic	Iqaluit	1	0
19	Spiderwort Suite	Nunavut	1	0
20	Heath Haven	Yukon	1	0
21	Geranium Gallery	Northwest Territories	1	0
22	Daffodil Domain	Hamilton	1	0
23	Lavender Loft	Kitchener	1	0

Result 1 x

Output

Action Output

Time Action Message Duration / Fetch

1 04:27:14 Select s.ShopID, s.ShopName, s.Location, Count(Distinct o.OrderID) As TotalOrdersPlaced, IFNULL(Sum(i.Qu... 10 row(s) returned 0.016 sec / 0.000 sec

Diagnose: The query output shows a clear division among the high efficiency and emerging storefronts operating under stock limitations. Stores like the Toronto Floral Fantasy, the Vancouver Blossom Boutique and the Montreal Petal Palace occupy the top spots with 2 transactions each despite total local inventories sitting at a high 70, 110, and 150 units, respectively. While remote expansion points such as the Whitehorse Dogbane Den, the Iqaluit Arum Attic and the Nunavut Spiderwort Suite have successfully logged initial order activity despite registering exactly 0 units in active inventory. This pattern shows that customer demand is scaling into new regions significantly faster than physical supply chains can stock shelves.

Strategize: To support rapid geographic expansion without facing critical supply shortfalls, management must adopt a swift fulfillment model for these inventoryless storefronts. For remote territories registering zero local items but holding active transaction demand, logistics should establish regional hubs or drop shipping systems from nearby high volume warehouses rather than investing immediately in heavy local retail infrastructure.

Part 3

Phase 1:

To verify the changes to the database were sufficient I ran queries to join tables, filter out specific items, and look at store groupings. The main challenge was dealing with how the data connects when trying to find average stock levels and comparing sales activity against low inventory stores in the expanded areas. To fix this I had to isolate the messy numbers into sub queries before running the final calculations to keep everything accurate. The AI tool Gemini (v1.5 Pro) helped by simplifying the parts of the code and troubleshooting the logic where the database queries got complex.

Phase 2:

The SQL queries helped the company to have an evidence based expansion strategy. For example inventory tracking highlighted that emerging territorial storefronts like Whitehorse and Iqaluit are successfully generating market demand and order transactions despite carrying zero active inventory. Rather than funding a capital intensive brick and mortar setups in these regions executives can confidently deploy a lean regional fulfillment model to serve these areas remotely.

The data also revealed two high value customer and logistics patterns critical for stakeholders. First a highly mobile, cross shopping consumer segment does exist. With specific loyalists visiting up to five distinct retail locations. Secondly core inventory density is heavily consolidated within just six main flower families. This may expose the business to major stockout risks on unstocked catalog items if supply disruptions occur or if there is ever a surge in demand.

A potential schema limitation that can impact the definiteness of these findings is that the database lacks historical data, customer home addresses, and fulfillment transit times. Without knowing whether cross shop traffic stems from consumer travel patterns or local stock shortages forcing clients to commute, leadership cannot perfectly separate genuine regional brand affinity from forced supply chain workarounds.

Phase 3:

To improve future predictive analytics a potential upgrade to the FlowerDB schema is implementing a `historical_inventory_logs` table that captures stock quantities at regular time stamped intervals alongside a customer table field tracking home postal codes. Additionally moving beyond snapshot views the data model should be allowed to run predictive seasonal demand forecasting and pinpoint whether cross shop habits stem from customer proximity or localized deficits.

This project strengthened my understanding of the Data Analyst's role showing that technical output can be directly translated to operational value. Analysts do not just verify data migration

or write clean code they can bridge raw data to form company changing decision making, helping logistics teams systematically optimize resource allocation and limit financial risk.

Technically this assignment has enhanced my ability to structure complex multi level nested queries and evaluate aggregated data groups under tight logical constraints. Translating those query results into strategic plans helped my business intelligence reporting skills which are easily transferable to any data driven decision making positions in a professional organization.